

Task 2 — breaking it on purpose

Anvar · Lab 1

Compiled each one with `g++ <file>.cpp -o <out>`. Same errors show up in VS Code's terminal.

Error 1 — forgot a semicolon

Deleted the `;` at the end of line 4.

```
break1_missing_semicolon.cpp: In function 'int main()':
break1_missing_semicolon.cpp:4:46: error: expected ';' before 'return'
   4 |         std::cout << "Hello, Anvar!" << std::endl
       |                                         ^
       |                                         ;
   5 |         return 0;
       |         ~~~~~
```

What happened: C++ needs a `;` to know a line is done. Without it, the compiler just keeps reading into the next line and gets confused there instead — so the error points at `return`, not at the actual missing semicolon. Classic off-by-one-line error message.

Error 2 — used a variable I never made

Typed `name` in the cout line but never declared it anywhere.

```
break2_undeclared_variable.cpp: In function 'int main()':
break2_undeclared_variable.cpp:4:31: error: 'name' was not declared in this scope;
did you mean 'tzname'?
   4 |         std::cout << "Hello, " << name << "!" << std::endl;
       |                               ^~~~
       |                               tzname
```

What happened: you can't use a variable before declaring it — the compiler has no idea what `name` is supposed to be. It even offers `tzname` as a "did you mean," which is some random system symbol that just happens to look similar. Not helpful here, but funny.

Error 3 — missing closing brace

Deleted the `}` that closes `main()`.

```
break3_mismatched_brace.cpp: In function 'int main()':
break3_mismatched_brace.cpp:5:14: error: expected '}' at end of input
    5 |         return 0;
      |                     ^
break3_mismatched_brace.cpp:3:12: note: to match this '{'
    3 | int main() {
      |           ^
```

What happened: every `{` needs its own `}`. The file just ends while the compiler still thinks it's inside `main()`, so it can't point at one broken line — instead it jumps back to show you exactly which `{` never got closed.